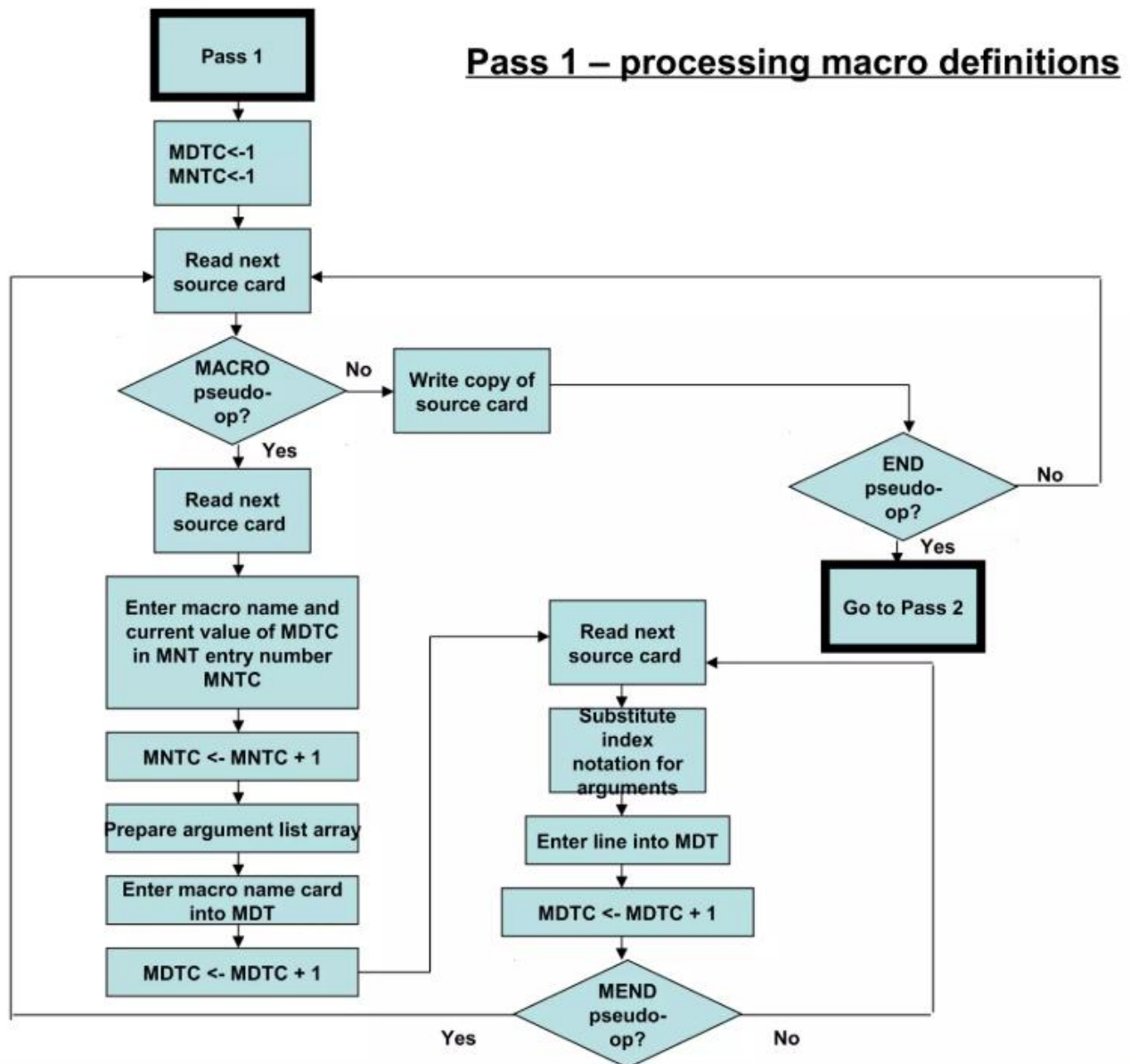


Chapter 3

1) Draw the flowchart of one Pass Macro.



A one-pass macro processor processes the program in a single scan.

When a macro definition is found, it is stored in MNT (Macro Name Table) and MDT (Macro Definition Table).

When a macro call occurs, it is expanded immediately using stored definitions and written to output.

2)Describe Macro Database.

Databases required for pass 1

MDT(Macro definition Table): To store body of macro definition.

MDTC(macro definition table counter):

To mark next available entry in MDT.

MNT(macro name table): used to store name of the macro.

MNTC(macro name table counter):

used to indicate next available entry in MNT.

ALA(Argument List array): to store an arguments.

Databases required for pass 2

1.COPY FILE

It is a file it contains the out put given from PASS1

2.MNT

It is used for recognizing macro name

3.MDT

It is used to perform macro EXPANSION

4.MDTP

It is used to point to the index of MDT .

The starting index is given by MNT

5.ALA

It is used to replace the index notation by it actual value

6.ESC

It is used to contain the expanded macro call which is given to the assembler for further processing

Chapter 4

1) Explain Direct Linking Loader with Suitable example.

Direct linking loader

- It is type of relocatable loader.
- It cant have direct access to the source program.
- To place the object program in memory there are 2 situations:
- Either the address of object code should be absolute which can be directly placed in memory or address is relative then it is the assembler who informs the loader about relative address.

Assembler should give following information to loader:

The length of segment

The list of external symbols

information about address constant

machine code translation of source program.

Assembler produces 4 types of card on object program:

1. **External symbol directory(ESD)**- contains information about all symbols that are defined in the program.
1. **Text Card(TXT)**- contains the actual object code translated versions of the source program.
1. **Relocation and linkage directory(RLD)**- contains information about those locations in the program whose contents depend on the address.
4. **END card-indicate** end of the object program.

External Symbol Dictionary (ESD)

- ESD record contain information about all the symbols that are defined in this program. But may be referenced elsewhere and all symbols referenced in this program but defined elsewhere.

It consist of three types of definition

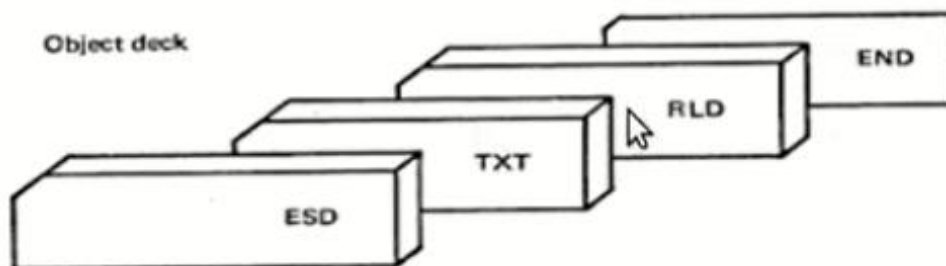
- SD- Segment Definition
- LD – Local Definition
- ER – External Reference

Example of object deck in Direct Linking Loaders

Source card Reference	Name	Type	ID	Relative Address	Length	ESD Card

Source card Reference	Relative Address	Content	Comment	TXT Card

Source card Reference	ESD ID	Length(bytes)	Flag + or -	Relative Address	RLD Card



Example of Direct Linking Loaders

Source Card Reference	Relative Address	Sample Program(Source deck)
1	0	PG1 START
2		ENTRY PG1X,PG1Y
3		EXTRN PG2X, PG2
4	20	PG1X ----
5	30	PG2Y ----
6	40	DC A(PG1X)
7	44	DC A(PG1Y + 15)
8	48	DC A(PG1Y - PG1X - 3)
9	52	DC A(PG2)
10	56	DC A(PG2X + PG2 - PG1X + 4)
11		END

Sample procedure PG1

ESD Card

Source card Reference	Name	Type	ID	Relative Address	Length
1	PG1	SD	01	0	60
2	PG1X	LD	--	20	--
2	PG1Y	LD	--	30	--
3	PG2	ER	02	--	--
3	PG2X	ER	03	--	--

TXT Card

Source card Reference	Relative Address	Content	Comment
6	40-43	20	
7	44-47	45	=30+15
8	48-51	7	=30-20-3
9	52-55	0	Unknown to PG1
10	56-59	-16	=-20+4

RLD Card

Source card Reference	ESD ID	Length(bytes)	Flag + or -	Relative Address
6	01	4	+	40
7	01	4	+	44
9	02	4	+	52
10	03	4	+	56
10	02	4	+	56
10	01	4	-	56

Loaders Scheme

Example of Direct Linking Loaders

Source Card Reference	Relative Address	Sample Program(Source deck)
12	0	PG2 START
13		ENTRY PG2X
14		EXTRN PG1X, PG1Y
15	16	PG2X ----
16	24	DC A(PG1X)
17	28	DC A(PG1Y + 15)
18	32	DC A(PG1Y - PG1X - 3)
19		END

Sample procedure PG2

ESD Card

Source card Reference	Name	Type	ID	Relative Address	Length
12	PG2	SD	01	0	36
13	PG2X	LD	--	16	--
14	PG1X	ER	02	--	--
14	PG1Y	ER	03	--	--

TXT Card

Source card Reference	Relative Address	Content	Comment
16	24-27	0	
17	28-3	15	Unknown address of PG1X+15
18	32-35	-3	Unknown to PG1Y- Unknown to PG1X -3

RLD Card

Source card Reference	ESD ID	Length(bytes)	Flag + or -	Relative Address
16	02	4	+	24
17	03	4	+	28
18	03	4	+	32
18	02	4	-	32

2) State advantages and disadvantages of relocating loader.

Relocating Loader

Advantages

1. **Program can be loaded anywhere**
No need for fixed memory location.
2. **Better memory utilization**
Uses available memory efficiently.
3. **Supports multiprogramming**
Multiple programs can run by placing them in different memory areas.
4. **Flexible execution**
Same program can run at different memory locations without modification.

Disadvantages

1. **More complex than absolute loader**
Requires relocation logic.
2. **Extra processing time**
Address modification during loading increases load time.
3. **Needs relocation information**
Object program must contain relocation bits.
4. **Slight memory overhead**
Additional data structures are required.

Advantages:

- Can be loaded anywhere
- Better memory use
- Supports multiprogramming

Disadvantages:

- Complex
- Slower loading
- Requires relocation info

3) State advantages and disadvantages of Direct Linking loader.

Direct Linking Loader (DLL)

Advantages

- 1. Supports Multiple Modules**
Can link different program modules and libraries together.
- 2. Efficient Memory Usage**
Only required modules are loaded into memory.
- 3. Faster Execution**
Linking and loading are done together, reducing overhead.
- 4. Dynamic Linking Support**
Allows use of shared libraries → saves space.
- 5. Flexibility**
Easy to modify or update individual modules without recompiling entire program.

Disadvantages

- 1. Complex Design**
More complicated than absolute or relocating loaders.
- 2. Slower Loading Time**
Linking during loading increases load time.
- 3. Error Handling Difficult**
Errors in external references can be harder to debug.
- 4. Requires More Memory**
Needs extra tables (MNT, MDT, etc.) during linking.
- 5. Dependency Issues**
Missing libraries/modules can cause execution failure.

Advantages:

- Supports multiple modules
- Efficient memory usage
- Provides flexibility

Disadvantages:

- Complex design
- Slower loading
- Dependency problems

4)State the function of a loader? Enlist the loader schemes.

Functions of Loader

Allocation- It calculates the size of the programme and allocates memory space for it. This is referred to as allocation..

Relocation- Modifies the object program so that it can be loaded at an address Different from the location originally specified.

Loading: It brings object program into the memory for execution.

Linking- Which combines two or more separate object programs &combine them into single executable file. (.exe)

loader schemes

❑ **Absolute Loader**: Loads program at a fixed address, no relocation.

❑ **Relocating Loader**: Adjusts addresses based on memory location.

❑ **Direct Linking Loader**: Performs linking + relocation for multiple modules.

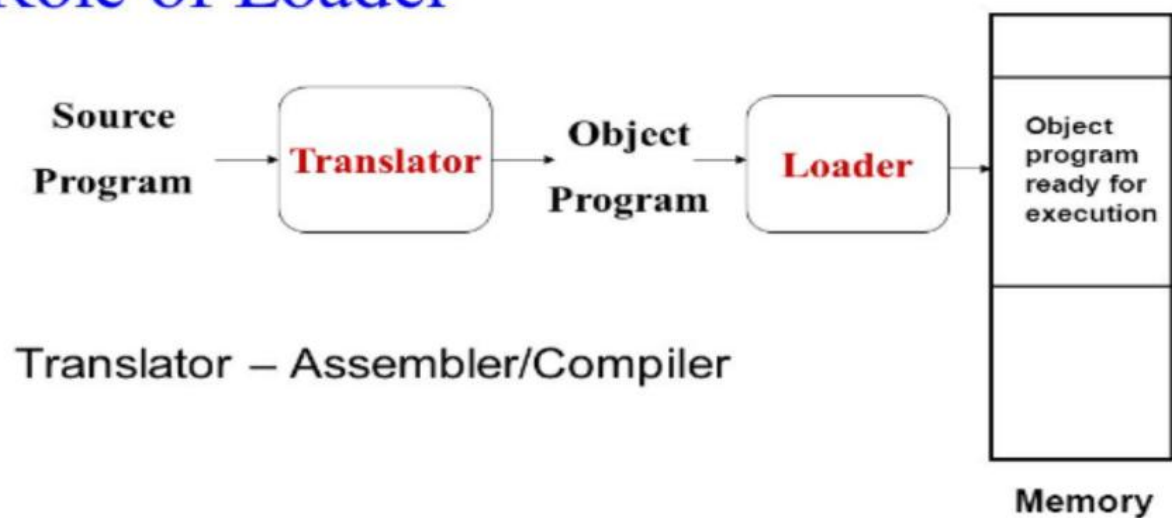
5) Define Relocation and Loader.

Relocation- Modifies the object program so that it can be loaded at an address Different from the location originally specified.

LOADER

- It is a part of OS that is responsible for loading programs into main memory.

Role of Loader



Chapter 5

1)Design Syntax tree for given expression.

2)Calculate left recursion from the following grammar.

3)Calculate FIRST and FOLLOW for the given grammar.

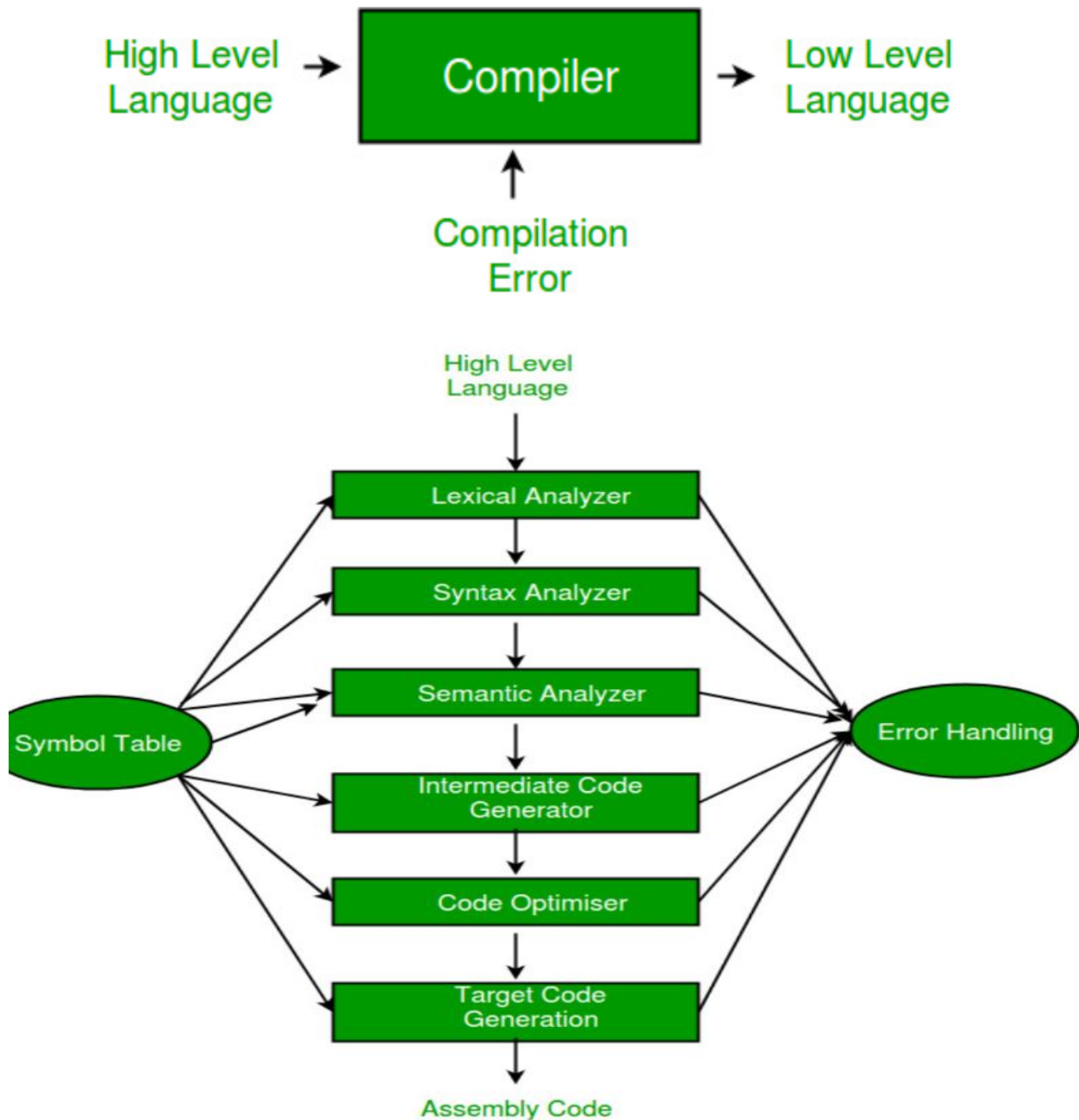
4)Differentiate Top-down and Bottom-up parser.

Sr.No.	TOP DOWN PARSING	BOTTOM UP PARSING
1	Process Starts with Root	Process Starts with Leaves
2	It starts with starting symbol of the grammar	It ends with starting symbol of the grammar
3	This parsing technique uses Left Most Derivation.	This parsing technique uses Right Most Derivation.
4	It is categorise by Recursive descent parser and Predictive parser	It is categorise by Operator precedance parser and Shift reduce parser
5	It is not accepting Ambiguous Grammar	It's accepting Ambiguous Grammar
6	It is less powerful as compare to bottom up parser	It is more powerful as compare to top down parser
7	It is simple to produce parser	It is Difficult to produce parser
8	It uses LL(1) grammar to perform parsing	It uses SLR,CLR, LALR grammar to perform parsing
9	Error detection is weak	Error detection is strong

5) Explain the phases of Compiler with example.

Compiler

It is a program that converts source code by programmer to machine level language.



Phase 1: Lexical Analysis

Lexical Analysis is the first phase of a compiler, also called a scanner.

It reads the source code character by character and converts it into tokens.

Tokens are the smallest meaningful units of a program such as:

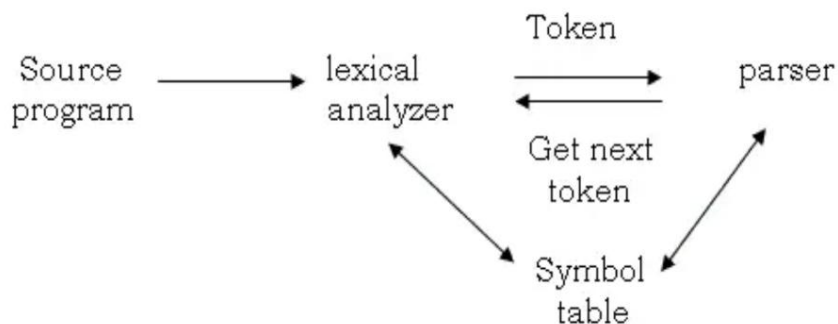
- Keywords
- Identifiers
- Constants
- Strings
- Operators
- Punctuation symbols

The lexical analyzer also:

- Removes spaces and comments
- Stores token information in the **symbol table**
- Passes tokens to the next phase (syntax analysis)

What is a Token?

A token is a group of characters that represents a single meaningful unit in a program.



Example:
x = y + 10

Tokens

x	identifier
=	Assignment operator
y	identifier
+	Addition operator
10	Number

Phase 2: Syntax Analysis (Parsing)

Syntax Analysis is the second phase of a compiler, also called a **parser**.

This phase checks the **structure of the program** using the tokens received from the lexical analyzer.

It creates a **tree-like structure** called a **parse tree**, which shows how the program is organized according to grammar rules.

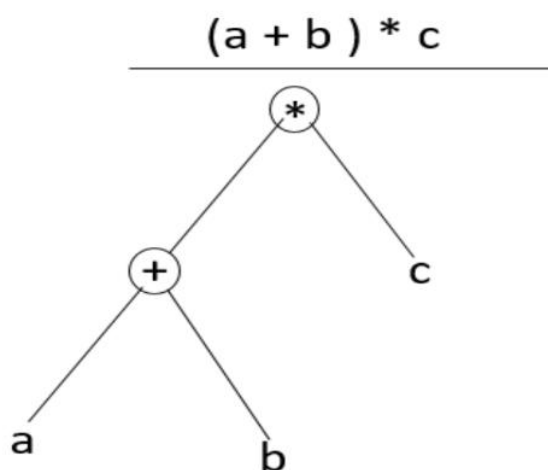
🔑 Main Purpose

The main goal of syntax analysis is to check whether the **source code is syntactically correct** (i.e., follows the rules of the programming language).

📌 Tasks Performed

- Receives tokens from the lexical analyzer
- Checks whether the program follows correct grammar rules
- Detects and reports syntax errors
- Builds a **parse tree** (hierarchical structure)

Consider parse tree for the following example
 $(a+b)*c$



Phase 3: Semantic Analysis

Semantic Analysis checks the **meaning of the program** and ensures it is logically correct.

It uses the **parse tree and symbol table** to verify correctness.

Key Function

- Performs **type checking** (operands must match operators)

Semantic Errors

- Type mismatch
- Undeclared variables
- Misuse of reserved keywords

Example:

```
float x = 10.1;
```

```
float y = x*30;
```

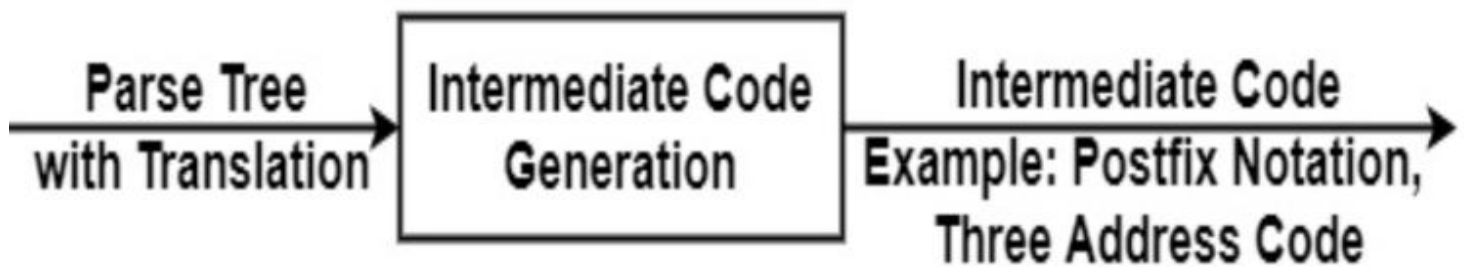
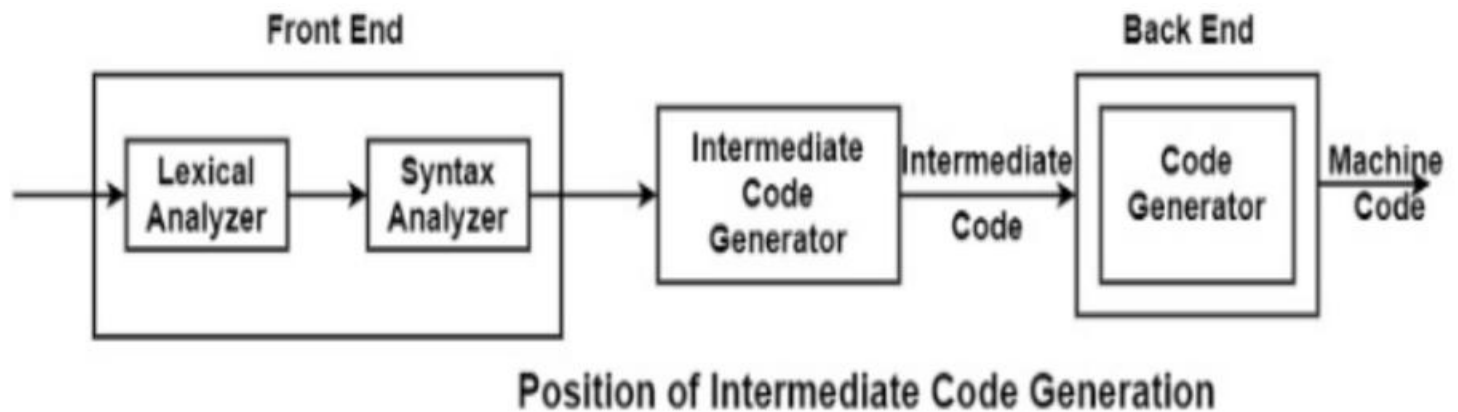
In the above example integer 30 will be typecasted to float 30.0 before multiplication, by semantic analyzer.

Phase 4: Intermediate Code Generation

After semantic analysis, the compiler generates **intermediate code (IR)**.

This code is a **middle form** between high-level language and machine code.

It takes input from the **annotated syntax tree** and converts it into a simpler form like **postfix or three-address code**.



Intermediate Code Generation

Postfix Notation

In postfix notation, the operator comes after an operand, i.e., the operator follows an operand.

Example

Postfix Notation for the expression $(a+b) * (c+d)$ is $ab + cd + *$

Phase 5: Code Optimization

Code Optimization improves the **intermediate code** to make the program **faster and more efficient**.

It removes **unnecessary code** and rearranges statements for better performance.

Goal

- Reduce execution time
- Reduce memory usage

Example:

Consider the following code

```
a = intofloat(10)
```

```
b = c * a
```

```
d = e + b
```

```
f = d
```

Can become

```
b = c * 10.0
```

```
f = e + b
```

Phase 6: Code Generation

Code generation is the **final phase of a compiler**.

- It takes input from the **code optimization phase**.
- It **produces object code** (machine-level code) as output.
- It **allocates memory locations** for variables.
- It **converts intermediate code into machine instructions**.

👉 The generated code must be **efficient in terms of speed and memory usage**.

Example (Simple)

Intermediate Code:

```
t1 = a + b
t2 = t1 * c
```

Generated Machine Code (Pseudo):

```
LOAD R1, a
ADD R1, b
STORE t1
LOAD R2, t1
MUL R2, c
STORE t2
```

Chapter 6

1) Explain Triples with example.

Triples in Intermediate Code

Triples is a representation of intermediate code used in compilers. It avoids using temporary variable names and instead refers to results by their **position (index)** in the list.

Structure of Triples

Each triple has **3 fields**:

- 1. **Operator (op)**
- 2. **Argument 1 (arg1)**
- 3. **Argument 2 (arg2)**

👉 Results are referred using the **index number** of the triple.

Example

Expression:

a = b + c * d



Step 1: Convert to Intermediate Form

t1 = c * d
t2 = b + t1
a = t2



Step 2: Triples Representation

Index	Operator	Arg1	Arg2
0	*	c	d
1	+	b	(0)
2	=	a	(1)

👉 (0) means result of index 0

👉 (1) means result of index 1

2) Explain any five code optimization techniques in compiler designing with suitable example.

Code Optimization Techniques in Compiler Design

Code optimization improves program efficiency by reducing execution time and memory usage.

1. Constant Folding

👉 Compile-time evaluation of constant expressions.

Example:

```
x = 5 + 10;
```

Optimized:

```
x = 15;
```

2. Constant Propagation

👉 Replaces variables with their constant values.

Example:

```
a = 10;
```

```
b = a + 5;
```

Optimized:

```
b = 10 + 5;
```

3. Dead Code Elimination

👉 Removes code that does not affect program output.

Example:

```
x = 10;
```

```
y = 20;
```

```
return x;
```

Optimized:

```
x = 10;
```

```
return x;
```

4. Common Subexpression Elimination

👉 Avoids recomputing same expressions.

Example:

```
t1 = a + b;
```

```
t2 = a + b;
```

Optimized:

```
t1 = a + b;
```

```
t2 = t1;
```

5. Loop Optimization (Loop Invariant Code Motion)

👉 Moves constant computations outside loop.

Example:

```
for(i = 0; i < n; i++) {  
    x = a + b;  
}
```

Optimized:

```
x = a + b;  
for (i = 0; i < n; i++) {  
}
```

5. Strength Reduction-

In this technique,

- As the name suggests, it involves reducing the strength of expressions.
- This technique replaces the expensive and costly operators with the simple and cheaper ones.

Example-

Code Before Optimization	Code After Optimization
$B = A \times 2$	$B = A + A$

1. Constant Folding CF

2. Constant Propagation CP

3. Dead Code Elimination DCE

4. Common Subexpression Elimination CSE

5. Loop Optimization (Loop Invariant Code Motion) LO

6. Strength Reduction SR

3) Describe the different ways of representing Intermediate Code? Explain with suitable example.

Intermediate codes can be represented in a variety of ways

The intermediate code can be represented in the form of postfix notation, syntax tree, three address codes, Quadruples, and triples.

Postfix Notation

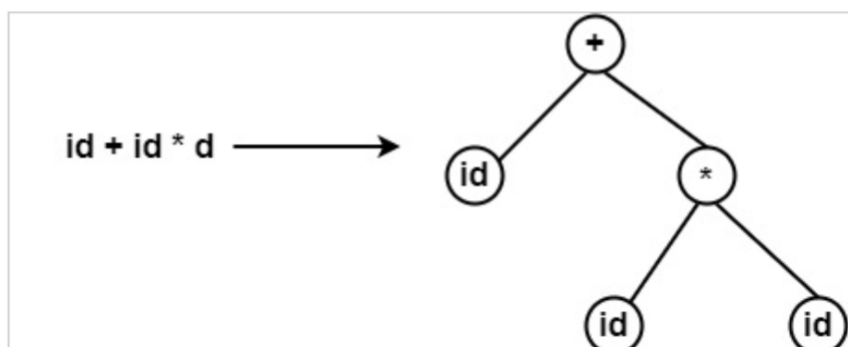
In postfix notation, the operator comes after an operand, i.e., the operator follows an operand.

Example

Postfix Notation for the expression $(a+b) * (c+d)$ is $ab + cd + *$

Syntax Trees

It is condensed form of parse tree in which leaves are identifiers and interior node will be operators.



Three Address Code–

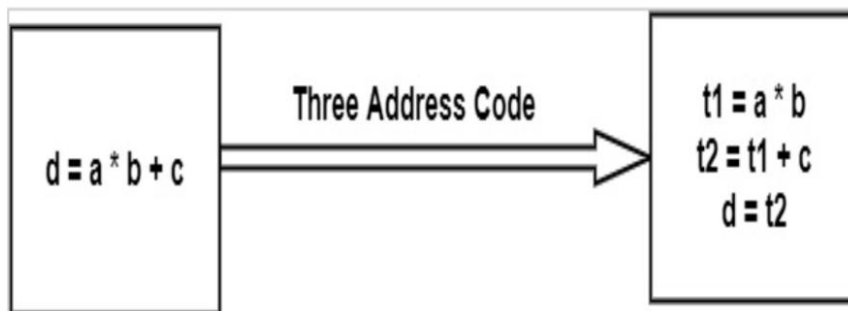
These are statements of form $c = a \text{ op } b$, i.e., in which there will be at most three addresses, i.e., two for operands & one for Result.

Each statement has the general form of:

Syntax:

$$\text{result} := \text{operand}_1 \text{ operator operand}_2$$

Example of three address code for the statement



Three-address code can be represented in two forms:

quadruples and triples.

1. Quadruples

👉 Each instruction has **4 fields**:

- Operator (op)
- Argument1 (arg1)
- Argument2 (arg2)
- Result

Example

Index op arg1 arg2 result

0 * b c t1

1 + a t1 t2

2 = t2 — x

👉 Uses **temporary variables (t1, t2)**

2. Triples

👉 Each instruction has **3 fields**:

- Operator
- Argument1
- Argument2

👉 Result is referred by **index number**

Example

Index op arg1 arg2

0 * b c

1 + a (0)

2 = x (1)

👉 (0) → result of index 0

👉 (1) → result of index 1

Key Difference

- Quadruples → Uses temporary variables
- Triples → Uses index references (no temporaries)

4) Explain different issues in the code generation phase of Compiler.

Issues in Code Generation

The main goal of a code generator is to produce **correct and efficient machine code**.

The following issues arise:

1. Input to Code Generator

👉 The code generator takes input from the **intermediate code (IC)** and **symbol table**.

- Intermediate code tells *what operations to perform*
- Symbol table gives *information about variables (type, size, location)*

Example:

t1 = a + b

👉 Code generator must know:

- a, b → variables
 - t1 → temporary value
-

2. Target Program

👉 The output should be suitable for the **target machine (CPU)**.

- Can be **assembly code** or **machine code**
- Must follow machine's instruction set

Example:

a = b + c

👉 For one machine:

LOAD R1, b

ADD R1, c

STORE a

👉 Another machine may use different instructions

3. Memory Management

👉 Deciding where variables and data will be stored.

- Stack (local variables)
- Heap (dynamic memory)
- Registers (fast access)

Example:

```
int a = 10;
```

👉 Compiler decides:

- Store a in register or memory location
-

4. Instruction Selection

👉 Choosing the best machine instructions.

- Many ways to perform same task
- Choose fastest/shortest one

Example:

```
ADD R1, 1
```

can be replaced by:

```
INC R1 (faster)
```

5. Register Allocation

👉 Deciding which variables go into registers.

- Registers are limited
- Important variables should be kept in registers

Example:

```
x = a + b;
```

```
y = x + c;
```

👉 Keep x in register to avoid memory access again

6. Evaluation Order

👉 Deciding order of evaluating expressions.

- Affects register usage and performance

Example:

$(a + b) + (c + d)$

👉 Better evaluation:

- Compute $(a + b)$ first
- Then $(c + d)$
- Then add results

👉 This reduces register usage and improves efficiency

Short Conclusion (for exam)

Code generation faces issues like handling input, selecting instructions, managing memory, allocating registers, and deciding evaluation order to produce efficient machine code.

1. Input to Code Generator

2. Target Program

3. Memory Management

4. Instruction Selection

5. Register Allocation

6. Evaluation Order